Structuring LLM Arguments: A Logic of Argumentation Based Approach

Aaron Habyarimana

July 15, 2026
Version: CleanThesis 0.4.1 - JGU patches

Johannes Gutenberg University Mainz
FB08
Institute of Computer Science
Data Mining

Bachelorarbeit

Structuring LLM Arguments: A Logic of Argumentation Based Approach

Aaron Habyarimana

1. Reviewer Prof. Dr. Stefan Kramer
Institute of Computer Science - Data Mining
Johannes Gutenberg University Mainz

2. Reviewer Dr. Maike Paetzel-Prüsmann
Institute of Computer Science
Johannes Gutenberg University Mainz

Supervisors Derian Boer and Lennart Baur

July 15, 2026

Aaron Habyarimana
Structuring LLM Arguments: A Logic of Argumentation Based Approach
Bachelorarbeit, July 15, 2026
Reviewers: Prof. Dr. Stefan Kramer and Dr. Maike Paetzel-Prüsmann
Supervisors: Derian Boer and Lennart Baur

Johannes Gutenberg University Mainz
Data Mining
Institute of Computer Science
FB08
Staudingerweg 9
55128 Mainz

Abstract

This thesis presents an end-to-end pipeline that generates, structures, and formally evaluates Abstract Argumentation Frameworks from natural-language debate topics using large language models. Rather than treating attack detection as a binary decision, the system prompts a language model to produce typed attack candidates, each annotated with an attack type, namely rebuttal, undercutting, or undermining, a confidence score, and a natural-language justification. A two-stage confidence filter routes candidates to acceptance, rejection, or verification. Accepted attacks are reduced to binary facts and handed to a Prolog solver that computes grounded, preferred, and stable extensions under Dung semantics. Evaluated on eight topics from the UKP Sentential Argument Mining Corpus, five of eight frameworks admit several preferred and stable extensions, each corresponding to an internally coherent position that a standalone language model leaves implicit. An ablation on identical atomized argument sets shows that typed and binary prompting extract substantially different attack relations, and that the multiplicity of extensions arises entirely through the symmetrization of rebuttals, which only typed extraction makes possible. An evaluation against a manual reference annotation of 201 argument pairs on one topic, created under written guidelines on a frozen argument set, shows that the typed extraction is measurably closer to this annotation than the binary baseline. It recovers three quarters of the annotated attacks and reaches a directed F1 of 0.615 against 0.458.

Abstract (german)

Diese Arbeit stellt eine durchgängige Pipeline vor, die mithilfe großer Sprachmodelle aus natürlichsprachlichen Debattenthemen abstrakte Argumentationsframeworks erzeugt, strukturiert und formal auswertet. Statt die Angriffserkennung als binäre Entscheidung zu behandeln, erzeugt das System typisierte Angriffskandidaten, die jeweils mit einem Angriffstyp (Rebuttal, Undercutting oder Undermining), einem Konfidenzwert und einer Begründung versehen sind. Ein zweistufiger Konfidenzfilter leitet die Kandidaten zur Annahme, Ablehnung oder Verifikation. Angenommene Angriffe werden auf binäre Fakten reduziert und an einen Prolog-Solver übergeben, der grounded, preferred und stable Extensions unter Dung-Semantik berechnet. In der Auswertung auf acht Themen des UKP-Korpus besitzen fünf von acht Themen mehrere preferred und stable Extensions, die jeweils einer in sich

konsistenten Position entsprechen, welche ein einzelnes Sprachmodell nur implizit lässt. Eine Ablation auf identischen atomisierten Argumentmengen zeigt, dass typisiertes und binäres Prompting deutlich verschiedene Angriffsrelationen extrahieren und dass die Mehrfach-Extensions vollständig durch die Symmetrisierung der Rebuttals entstehen, die nur die typisierte Extraktion ermöglicht. Eine Auswertung gegen eine manuell annotierte Referenz aus 201 Argumentpaaren auf einem Thema, erstellt nach schriftlichen Richtlinien auf einer eingefrorenen Argumentmenge, zeigt, dass die typisierte Extraktion dieser Annotation messbar näher kommt als die binäre Baseline. Sie findet drei Viertel der annotierten Angriffe und erreicht einen gerichteten F1 von 0,615 gegenüber 0,458 für die Baseline.

Contents

1 Introduction

1.1 Motivation

Human argumentation is hard to capture with the rules of classical logic alone. Instead of deriving truth from fixed premises, a debate proceeds through arguments and counter-arguments, where a claim can be accepted or defeated depending on which other claims stand against it rather than being simply true or false. The same difficulty reappears wherever an automated system must decide which of several conflicting claims can be held together, most visibly in language models, whose fluent output can assert a conclusion that the reasons given do not support.

A concrete debate makes the difficulty tangible. Asked about nuclear energy, people argue that nuclear power is nearly carbon-free, that meltdowns are statistically rare, that waste storage remains an unresolved safety problem, that plants are potential terrorism targets, and that subsidies for nuclear power crowd out investment in renewables. These claims attack each other in specific and directed ways, and typically more than one internally coherent position survives, such as a pro camp built around climate benefits and a contra camp built around catastrophic risk. Which claims can be held together consistently is a structural question about the debate, and answering it requires making the attack relationships explicit.

One established way to formalise arguments is the logic of argumentation [10], which provides a language for representing the structure of reasons that support or undercut a line of reasoning. What such formalisms leave open is where the arguments and their relations come from in practice, since the examples in the literature often appear constructed and providing the arguments frequently seems harder than the reasoning over them.

Large language models (LLMs) approach the problem from the opposite side. Where formal argumentation offers structure without content, LLMs produce content without structure. They can generate reasons for or against a cause in fluent natural language, but they lack the formal scaffolding that would allow any systematic analysis of how those reasons relate. A standalone LLM can be asked to summarise the positions in a debate, yet its answer is another ungrounded generation without a guarantee of internal consistency. The proposal of this thesis is to combine the two sides, using the LLM for what it is good at, namely producing arguments and identifying their attacks, and handing the resulting structure to a formal layer that is good at reasoning

about consistency. The formal layer makes explicit which sets of arguments can be jointly accepted, a property that the raw LLM output leaves implicit.

The natural target of such a pipeline is argument sets produced by an LLM or by any other source, since the pipeline itself is agnostic to where the arguments come from. The evaluation nonetheless draws its main argument sets from an annotated corpus rather than from generation. The reason is methodological. If the same model produced the arguments and also made the attacks between them explicit, the input would itself be an artefact of the model, and there would be no independent reference against which the extracted structure could be checked. Corpus arguments fix the input independently of the model and make the comparison against a manual annotation possible. A second setting on LLM-generated arguments then shows that the same pipeline runs end-to-end on generated input.

1.2 Research Questions and Contributions

This thesis builds and evaluates an end-to-end pipeline that turns natural-language debate text into a formally evaluated abstract argumentation framework (AF) in the sense of Dung [6]. An LLM proposes typed attack candidates between arguments, a confidence filter routes them to acceptance, rejection, or verification, and a Prolog solver computes grounded, preferred, and stable extensions over the accepted attacks.

The work is guided by four research questions.

RQ1 (Feasibility). Can an end-to-end pipeline be built that turns natural-language arguments into formal Dung extensions through LLM-based attack extraction and a symbolic solver?

RQ2 (Typed vs. binary extraction). Does *typed* attack prompting, which forces the model to name the attack type and justify it, produce different or more reliable attack relations than plain *binary* prompting, and how does this affect the resulting frameworks?

RQ3 (Extraction quality). How well do the extracted attacks match a manual reference annotation in terms of precision, recall, and F1 score, and which error types dominate?

RQ4 (Argumentative structure). What argumentative structures do the formal semantics reveal across the eight topics of the UKP corpus (Table 5.2), in particular regarding multiple extensions, and when does the formal layer add information beyond the grounded extension?

Along these questions the thesis makes four contributions: a complete and reproducible implementation of the pipeline, from the atomization of compound sentences into single claims through typed attack extraction and confidence

filtering to a Prolog solver (RQ1), an empirical analysis of the resulting frameworks over eight topics of the UKP Sentential Argument Mining Corpus [18] (RQ4), an ablation that compares typed against binary attack prompting on identical argument sets and separates the effect of the categorization from the effect of rebuttal symmetrization (RQ2), and an evaluation of the attack detection against a validated manual annotation of all 201 argument pairs of one topic (RQ3).

A condensed version of the pipeline and the cross-topic evaluation has been submitted to the International Joint Conference on Learning and Reasoning (IJCLR 2026). In addition to the material of that submission, this thesis contributes explicit research questions, an expanded background, the typed-versus-binary ablation, which the submission motivates but does not measure, and the manual reference evaluation for RQ3.

The remainder of the thesis is structured as follows. Chapter 2 introduces the formal background, namely Dung's abstract argumentation frameworks and their semantics, the attack typology of Pollock and ASPIC+, and the basics of argument mining and language models. Chapter 3 reviews related work on LLM-based argumentation pipelines and positions the thesis against it. Chapter 4 describes the pipeline step by step, together with the design decisions behind it. Chapter 5 presents the experimental setup and results, including the cross-topic evaluation, the typed-versus-binary ablation, the evaluation against a manually annotated reference, and a case study with an analysis of the recurring error types. Chapter 6 discusses what the formal layer adds, the limitations of the study, and the main design trade-offs, before Chapter 7 summarises the answers to the research questions and outlines future work.

2 Foundations

This chapter introduces the formal and conceptual background the thesis builds on. Section 2.1 defines Dung's abstract argumentation frameworks and the three semantics the pipeline computes. Section 2.2 introduces the attack typology of Pollock and ASPIC+ that the extraction step uses. Section 2.3 sketches argument mining, and Section 2.4 covers the aspects of large language models relevant to this work.

2.1 Abstract Argumentation Frameworks

Abstract argumentation frameworks (AFs), introduced by Phan Minh Dung in 1995 [6], are the most established formal model of argumentation. The framework is deliberately abstract. It treats arguments as atomic entities without internal structure and represents only the conflicts between them.

Definition 2.1 (Argumentation Framework)**.** An argumentation framework is a pair $AF = \langle \mathcal{A}, \mathcal{R} \rangle$, where $\mathcal{A}$ is a finite set of arguments and $\mathcal{R} \subseteq \mathcal{A} \times \mathcal{A}$ is a binary attack relation. We say that an argument a attacks an argument b if $(a, b) \in \mathcal{R}$.

The central question is which sets of arguments can rationally be accepted together. The following notions, all due to Dung [6], build up to an answer.

Definition 2.2 (Conflict-freeness and defense)**.** A set $S \subseteq \mathcal{A}$ is *conflict-free* if there are no arguments $a, b \in S$ with $(a, b) \in \mathcal{R}$. An argument a is *defended* by S if for every b with $(b, a) \in \mathcal{R}$ there is some $c \in S$ with $(c, b) \in \mathcal{R}$.

Definition 2.3 (Admissibility and characteristic function)**.** A conflict-free set S is *admissible* if it defends every one of its elements. The characteristic function $F_{AF} : 2^{\mathcal{A}} \to 2^{\mathcal{A}}$ maps a set to the arguments it defends, $F_{AF}(S) = \{a \in \mathcal{A} \mid S \text{ defends } a\}$.

On top of admissibility, Dung defines several *semantics*, each formalising a different standard of collective acceptability. The following notion of a complete extension is the common core of all semantics used in this thesis.

Definition 2.4 (Complete extension)**.** An admissible set S is a *complete extension* if it contains every argument it defends, that is, if $F_{AF}(S) = S$.

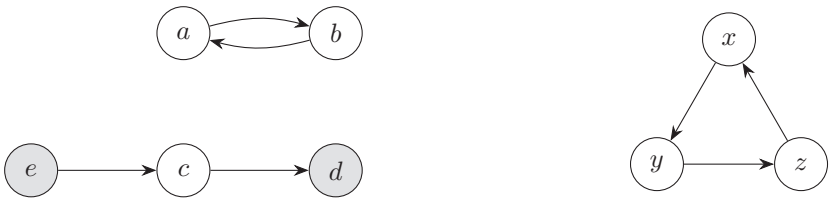

Fig. 2.1.: Two small argumentation frameworks. Left: the running example AF_1 with a mutual attack between a and b. Filled nodes form the grounded extension $\{d, e\}$, and the two preferred extensions $\{a, d, e\}$ and $\{b, d, e\}$ extend it in incompatible ways. Right: the odd-length cycle AF_2, whose only complete extension is the empty set and which has no stable extension.

A complete extension is a position that is both defensible and exhaustive. It contains nothing it cannot defend, and it leaves out nothing it could defend. This thesis computes three semantics that refine completeness, chosen to span a wide spectrum of strictness.

- The **grounded extension** is the least fixed point of F_{AF}, obtained by iterating F_{AF} from the empty set. It is the smallest complete extension, it is the most cautious of the semantics, it always exists, and it is unique.
- A **preferred extension** is a maximal admissible set. Every preferred extension is complete, and it represents the largest internally coherent position a debate supports. A framework can have several preferred extensions.
- A **stable extension** is a conflict-free set S that attacks every argument outside S. Stable extensions are the strictest of the three and are not guaranteed to exist.

The semantics form an inclusion hierarchy [6]. Every stable extension is preferred, every preferred extension is complete, and the grounded extension is contained in every complete extension. The last property matters twice in this thesis. Conceptually, the grounded extension is the uncontroversial core that every rational position must accept. Computationally, it is the basis of the solver optimization described in Section 4.6.

The grounded semantics collapses a debate to a single cautious set, whereas preferred and stable extensions, which may be multiple, reveal whether a topic supports several internally consistent positions. This distinction is central to the evaluation in Chapter 5.

A worked example

The difference between the three semantics is best seen on a small example. Consider the framework AF_1 on the left of Figure 2.1, with arguments $\mathcal{A} = \{a, b, c, d, e\}$ and attacks $\mathcal{R} = \{(a, b), (b, a), (e, c), (c, d)\}$. The structure mirrors a

debate in miniature. The arguments a and b are the cores of two opposing camps that directly contradict each other, for instance the claim from the UKP nuclear-energy topic that waste storage is safe and secure against the claim that nuclear waste storage remains an unresolved safety problem. Each holds only if the other fails, so the conflict runs both ways, which the mutual attack between a and b captures. The argument c raises an objection that attacks a further pro claim d, for instance that nuclear power is virtually carbon-free, and c is in turn refuted by e.

Iterating the characteristic function from the empty set yields $F_{AF}(\emptyset) = \{e\}$, since e is the only unattacked argument, and then $F_{AF}(\{e\}) = \{d, e\}$, since e defends d against its only attacker c. A third step adds nothing, so the grounded extension is $\{d, e\}$. It accepts the uncontroversial arguments and stays silent on the conflict between a and b. The preferred semantics goes further. Both $\{a, d, e\}$ and $\{b, d, e\}$ are admissible, since a and b defend themselves against each other, and neither set can be enlarged without a conflict, so the framework has two preferred extensions, one per camp, each containing the grounded extension as the inclusion hierarchy predicts. Both sets are also stable, since each attacks every outside argument. The example shows the division of labour that this thesis exploits. The grounded extension identifies the safe core, and the preferred and stable extensions make the incompatible ways of completing that core explicit.

The odd-length cycle AF_2 on the right of Figure 2.1 illustrates why stable extensions are not guaranteed to exist. No single argument in this example can defend itself against its attacker, so the empty set is the only admissible and hence the only complete and preferred extension. The empty set attacks nothing, so it is not stable, and AF_2 has no stable extension at all.

2.2 Attack Typology

Dung's framework is abstract in that it does not say *why* one argument attacks another. For the extraction step, however, the kind of attack matters. Pollock [16] distinguishes two kinds of defeaters in natural-language reasoning, rebutting defeaters that contradict a conclusion and undercutting defeaters that break the inference leading to it. ASPIC+ [13], a structured argumentation framework that builds Dung-style attack graphs from explicitly represented premises, inference rules, and conclusions, adds undermining as an attack on a premise and gives each attack a precise structural target. This yields the three-way typology that the extraction step of this thesis uses.

- A **rebuttal** contradicts the *conclusion* of the target. An example is the pair "nuclear power is clean" against "nuclear plants cause catastrophic contamination".

- An **undercutting** attack challenges the *inference* from premises to conclusion. The conclusion does not follow even if the premises hold. The claim “nuclear power is safe because its CO_2 emissions are low” is undercut by “low emissions say nothing about accident or waste risks”, which grants the premise but severs the step to the conclusion.
- An **undermining** attack hits a *premise* the target relies on, without engaging the conclusion directly. The claim “containment makes plants safe” is undermined by “radiation cannot be contained indefinitely”.

A property that matters for the construction of the framework is that a rebuttal is inherently mutual. If two conclusions negate the same proposition, then one is true only if the other is false, so the conflict holds in both directions. Without a preference or strength ordering, which Dung’s abstract framework does not provide, neither side takes precedence. The pipeline therefore symmetrizes rebuttals, in line with the treatment of rebuttals in ASPIC+ [13]. Undercutting and undermining attacks, by contrast, remain directed. This asymmetry has consequences for the cycles that appear in the resulting frameworks, which Chapter 5 discusses.

2.3 Argument Mining

Argument mining (AM) aims to automatically identify argumentative components, such as claims and premises, and the relations between them from natural-language text. Traditional approaches decompose the problem into subtasks including argument detection, stance classification, and relation prediction, and address them with supervised learning on annotated corpora [11]. More recently, LLMs have begun to replace task-specific classifiers with prompt-based methods that cover these subtasks in a single framework [12]. Most existing work, however, evaluates AM subtasks in isolation and stops before connecting the extracted relations to formal reasoning, which leaves the question of which arguments are collectively acceptable unanswered. This thesis targets exactly that gap.

2.4 Large Language Models

Large language models are neural networks trained on large text corpora to predict the next token, and through that objective they acquire a broad ability to follow instructions given in natural language. Two properties matter for this thesis. Model behaviour is steered through *prompts*, so the wording of the task directly shapes the output, and even small changes to a prompt can shift the results markedly [17]. This sensitivity is the very lever that the typed extraction of this thesis relies on, and at the same time a threat to stability, so

the pipeline versions every prompt and Section 6.2 discusses the observed effect of prompt revisions. A sampling *temperature* of zero in turn makes the output as deterministic as the model allows, but not absolutely deterministic, since server-side batching and numerical effects can still alter the sampled tokens.

3 Related Work

This chapter reviews work that combines language models with formal or relational argumentation and positions the present thesis against it. It first discusses end-to-end pipelines that turn text into argumentation frameworks, then work that uses LLMs specifically for relation extraction, the use of the UKP corpus, and finally related formalisms. In short, the thesis adopts the end-to-end perspective of the pipeline systems below and supplies the empirical evaluation that the closest prior systems leave undone.

3.1 LLM-based End-to-End Argumentation Pipelines

The closest work to this thesis is ARGUS [3], which combines LLM-based argument mining with the construction of argumentation frameworks and symbolic solvers in an end-to-end architecture. ARGUS shares the central idea pursued here, namely to let an LLM handle the noisy natural-language part of the problem and to delegate the acceptability question to a formal solver. It sets out the architecture and the Dung-semantics perspective but does not report an empirical evaluation of the resulting frameworks, so it leaves open how many attacks such a pipeline accepts, whether the resulting frameworks are cyclic, and how many extensions they admit. Chapter 5 answers exactly these questions.

ArgLLMs [7] also augments LLMs with formal argumentative reasoning but takes a quantitative route. Instead of asking which arguments can be jointly accepted, it builds Quantitative Bipolar Argumentation Frameworks (QBAFs) in which each argument carries a numerical strength that gradual semantics propagate along support and attack edges. The system is evaluated on binary claim verification, so its output is a single scalar per claim rather than a set of jointly admissible arguments, and its evaluation targets a downstream task rather than the structure of the argumentation itself.

A further recent example of an end-to-end LLM system for argument mining is the work of Pirozelli et al. [15], which applies LLMs to the argument mining subtasks within a single system rather than through separate task-specific classifiers. It reflects the broader trend that LLMs are becoming the default engine for extraction, but like most argument mining work it stops at the extracted structure and does not connect the result to Dung semantics or evaluate the collective acceptability of the mined arguments. The pipeline of this thesis differs from ArgLLMs and Pirozelli et al. by computing set-based

Dung extensions rather than numerical strengths or raw extracted relations, and from the Dung-based ARGUS by extracting typed, confidence-filtered attacks and evaluating the resulting frameworks empirically.

3.2 LLMs for Attack Relation Extraction

The extraction step of the pipeline, which classifies the relation between every pair of atomic arguments, is most directly related to work that studies LLMs on relation-based argument mining. Gorur et al. [8] ask precisely whether large language models can identify the relations, such as support and attack, that hold between arguments, the subtask on which the present pipeline depends. They find that suitably prompted LLMs are competitive with and often better than task-specific relation classifiers, which supports the design decision of this thesis to let an LLM propose the attack candidates. Their perspective remains diagnostic, in that they measure how well LLMs recover these relations in isolation, whereas the present thesis treats relation extraction as one stage of a larger pipeline and feeds its output into a formal solver. Two design choices set the extraction here apart. The LLM is asked not only whether an attack exists but also which of the three attack types of Section 2.2 it instantiates, which forces engagement with the argumentative structure rather than a surface judgment on opposing stances. Each decision moreover carries a confidence score that routes borderline cases to a separate verification prompt instead of accepting them outright. The value of the typed prompt over a plain binary one is measured directly in the ablation of Section 5.4.

3.3 Argument Mining on the UKP Corpus

The evaluation uses the UKP Sentential Argument Mining Corpus [18], which provides sentence-level stance annotations for eight controversial topics. Prior work of the supervising group applies knowledge-based graph argument mining with topic modeling to the same corpus [9]. Their task differs from the one studied here. They classify *single sentences* as argument or non-argument for a given topic, without stance and without any relation between arguments, and report an accuracy of 85 percent alongside an F1 score of 67 percent. Two lessons carry over to this thesis. The 18 point gap between their accuracy and their F1 illustrates how strongly a majority of negative instances inflates accuracy on this corpus, which is why the reference evaluation in Section 5.5 reports precision, recall, and F1 on the attack class rather than accuracy. Their F1 also gives a rough sense of how demanding the corpus is for a related sentence-level subtask, though it is not a directly comparable benchmark,

since pairwise attack detection is a different and arguably harder problem with a lower positive rate.

3.4 Related Formalisms

Most closely related on the symbolic reasoning side is the recent work of Alfano et al. [1], which connects LLM-based argument mining with argumentation and description logics, a family of decidable knowledge representation logics designed for ontological reasoning. Their system extracts a Fuzzy Argumentative Knowledge Base (FAKB) from raw text and combines it with a QBAF. This enriches the formal layer and makes it more expressive but also heavier, since it commits to a strength ordering that has to be justified. The present thesis deliberately stays with Dung's abstract framework instead, since the question of interest is which arguments can be jointly accepted rather than how strong each argument is, and without a preference ordering binary attack facts are the natural representation for it, as Chapter 4 argues in more detail. The typed attacks and confidence scores are not discarded but retained as metadata, which keeps the door open to a richer formalism such as ASPIC+ [13] or a QBAF as future work.

4 Method: A Pipeline from Debate Text to Formal Extensions

This chapter describes the pipeline that turns raw debate text into a formally evaluated Dung framework. Section 4.1 gives an overview, and the remaining sections describe the five steps in turn, each with the design decision behind it.

4.1 Overview

The pipeline runs in five steps, summarised in Figure 4.1. Step one acquires arguments and atomizes them into self-contained claims, step two proposes typed attack candidates for every pair of claims, step three routes the candidates by confidence and verifies the borderline cases, step four converts the accepted candidates into binary Dung attack facts, and step five solves the resulting framework under grounded, preferred, and stable semantics with a Prolog solver.

4.2 Step 1: Argument Acquisition and Atomization

Arguments enter the pipeline in one of three ways. They are generated from scratch by prompting an LLM to produce n pro and n con arguments on a topic, extracted from an existing text corpus, or loaded directly from an argument database. In all cases an argument is represented by a tuple of an identifier, a stance label, and the argument text.

Natural-language arguments frequently bundle several claims. A compound sentence such as “Although nuclear energy produces waste, it is cleaner than coal” mixes a concession with a main claim, and treating it as one unit would distort the attack relations extracted next. The pipeline therefore prompts an LLM to split each argument into self-contained atomic claims under three rules.

1. **Drop concessive fragments.** Every component introduced by “although”, “while”, “despite”, and similar markers is a rhetorical hedge rather than a standalone argument and is dropped.

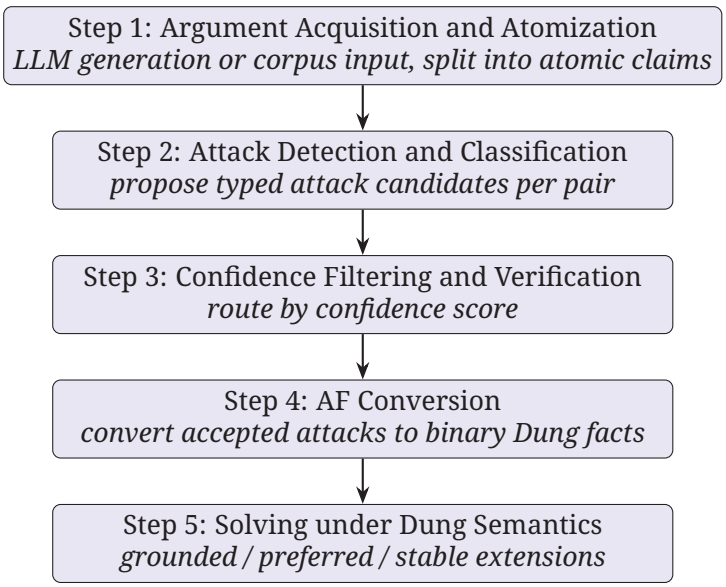

Fig. 4.1.: Overview of the pipeline, from raw debate text to formally evaluated Dung extensions in five steps.

2. **Resolve pronouns.** A pronoun that clearly refers to a specific subject is replaced by that subject. A fragment whose referent cannot be recovered is removed, so that every atom is understandable in isolation.
3. **Filter non-argumentative content.** A statement that neither supports nor opposes the claim under debate is dropped.

A fourth rule keeps the stance intact. Reported or conceded positions of the opposing side, such as “opponents argue X”, are not promoted to their own atom. Read in isolation, such a fragment states a claim of the opposing side while carrying the stance label of its speaker, so the extraction step would treat it as a claim the speaker actually holds. Every attack decision that involves such an atom would then rest on a wrongly attributed claim, for instance a con argument attacking a supposed pro atom that no pro speaker asserts.

A concrete case from the evaluation illustrates the step. The contra sentence on meltdown statistics states that in two out of three meltdowns there would be no deaths, in one out of five there would be over a thousand deaths, and in one out of one hundred thousand there would be fifty thousand deaths, and thus bundles three separable claims. Atomization splits it into three atoms, each carrying one scenario, so that the extraction step can judge each claim on its own. Each argument is atomized independently, which allows parallelization. The resulting set of atomic arguments is denoted $\mathcal{A}'$ and forms the input to the next step. Every atom keeps the identifier of its source argument. The first atom keeps the bare identifier and every further atom appends a letter suffix, so that for instance `pro5` can yield the atoms `pro5` and `pro5b`, which share the base `pro5`. The first atom deliberately does not receive a suffix of its own, so that an argument that needs no splitting passes

through the step with its identifier unchanged. The full atomization prompt is given in Appendix A.1.

4.3 Step 2: Attack Detection and Classification

With the atomic arguments in place, the pipeline examines every pair in $\mathcal{A}'$ for a potential attack, regardless of stance, since same-stance arguments can also conflict. Two pro-nuclear arguments may, for example, prioritise incompatible safety strategies. Pairs of atoms that share a base are skipped, since they originate from the same source argument.

Rather than a binary decision, the pipeline prompts the LLM to detect and categorize each potential attack at once and to return a confidence score and a justification. Asking for the type rather than a mere yes or no forces the model to engage with the argumentative structure of the pair instead of pattern matching on opposite stances, and the score and justification make each decision traceable and provide a routing signal for the next step. Each candidate carries four fields.

```
direction:   A_ATTACKS_B | B_ATTACKS_A | NONE
attack_type: rebuttal | undercutting | undermining | NONE
confidence:  integer in [0, 100]
reason:      one-sentence justification
```

The prompt provides the full definitions of the three attack types from Section 2.2 and enforces that if `direction` is `NONE` then `attack_type` is `NONE` and `confidence` is zero, which keeps the output internally consistent. Rebuttals are treated as symmetric, so the reverse edge is added automatically, as motivated in Section 2.2.

Beyond the type definitions, the prompt encodes two rules that emerged from inspecting misclassifications in early runs, and both target the rebuttal label, the only label whose misuse creates spurious cycles through symmetrization. The first rule demands a genuine contradiction. Two claims rebut each other only if they negate the same proposition, so that one can be true only if the other is false. Merely taking the opposing stance, addressing a different subtopic, or arguing for the other side is explicitly ruled out as a rebuttal. The second rule blocks a recurring category error. A descriptive or empirical claim, such as a statistic or a report of what people believe, and a normative claim about what ought to be done are not logically incompatible and therefore cannot rebut each other, although the descriptive claim may still undermine a premise of the normative one. Both rules restrict when the mutual, cycle-creating attack pattern may be asserted, and every prompt is versioned so that results are always tied to the exact wording that produced them. A final revision of the prompt, used by the reference evaluation run of Section 5.5, adds a third rule that applies to all three attack types. It rejects a pair whenever both specific claims can be true at the same time without tension, and it

requires an undermining attack to name the premise it hits. The full prompts are listed in Appendix A.1.

The remaining pairs are grouped into batches of size b, which reduces the number of API calls by a factor of b. Pairs for which the model returns malformed output are retried individually. Algorithm 1 summarises this classification step.

This typed prompt is exactly the component whose value the ablation in Section 5.4 measures, by comparing it against a binary variant that omits the attack type, the type definitions, and both rules above, and asks only whether an attack exists.

4.4 Step 3: Confidence Filtering and Verification

Not all candidates are equally reliable. The confidence score, which is an indication of the model's certainty rather than a calibrated probability, routes each candidate through a three-way decision with two thresholds θ_{acc} and θ_{ver}.

$$\text{decision}(c) = \begin{cases} \textbf{accept} & c \geq \theta_{\text{acc}} \\ \textbf{verify} & \theta_{\text{ver}} \leq c < \theta_{\text{acc}} \\ \textbf{reject} & c < \theta_{\text{ver}} \end{cases}$$

Borderline candidates in the verify region are re-examined by a dedicated verifier prompt that asks the model to confirm or reject the attack given the two arguments and the proposed type, but without access to the original confidence score or justification. Withholding both keeps the verifier from simply adopting the extractor's judgment instead of forming an independent one. Accepted and confirmed candidates pass to the next step, and rejected ones are discarded.

The thresholds are set to $\theta_{\text{acc}} = 70$ and $\theta_{\text{ver}} = 50$ by default. They are design choices made after inspecting the confidence values of early runs, in which the model concentrated its scores at round values and separated clear attacks, which typically received 80 or more, from speculative ones in the range around 50 to 65. The thresholds were deliberately not tuned against any evaluation data, since no annotated labels existed at design time and tuning them on the data used for the evaluation would leak information. The verification step exists precisely so that the acceptance threshold can stay high without silently losing every borderline case, because the middle band gets a second, independent judgment instead of a hard cut.

Algorithm 1 Typed attack candidate extraction

Require: atomized arguments $\mathcal{A}'$, topic T, batch size b
Ensure: set of typed attack candidates C

$pairs \leftarrow \{(a_i, a_j) \mid i < j,\ \text{base}(a_i) \neq \text{base}(a_j)\}$ ▷ exclude same-base pairs
$C \leftarrow \emptyset$
for each $batch \in \text{chunk}(pairs, b)$ **in parallel do**
    $raw \leftarrow \text{ClassifyTyped}(batch, T)$ ▷ direction, type, confidence, reason per pair
    if raw parses as a JSON list **then**
        $C \leftarrow C \cup raw$
    else
        retry the batch once, then classify missing pairs individually
    end if
end for
return C

4.5 Step 4: AF Conversion

Every accepted candidate is converted to a binary Dung fact $\texttt{att}(A, B)$, deliberately dropping the attack type from the formal semantics. Unlike gradual QBAF semantics, which assign each argument a numerical strength, Dung semantics ask which arguments can be jointly accepted, a set-based rather than numerical question, so binary attack facts are the natural representation. The attack type, the confidence score, and the justification are not lost. They are retained as comments and metadata and remain available for analysis and error inspection.

4.6 Step 5: Solving under Dung Semantics

The framework is solved by an AF solver implemented in Prolog under the three semantics of Section 2.1, using standard fixed-point and enumeration procedures [4]. The grounded extension is computed by iterating the characteristic function from the empty set in polynomial time. Computing preferred and stable extensions requires enumerating conflict-free sets, which is exponential in the worst case.

To keep this tractable the solver applies a *grounded-first reduction*. The underlying idea is not new. Computing the cheap grounded labelling first and restricting the expensive search to the arguments it leaves undecided is a standard simplification in computational argumentation [2, 5], and this section describes its concrete form in the pipeline. The reduction rests on the inclusion hierarchy from Section 2.1. Let G be the grounded extension and let U be the set of arguments that are neither in G nor attacked by G. This set is

called the *undecided core* in the following. The name comes from the labelling view of argumentation semantics [2], in which the grounded labelling marks the members of G as IN, the arguments attacked by G as OUT, and precisely the arguments in U as UNDEC, that is, undecided. Every complete extension, and hence every preferred and stable extension, contains G and excludes all arguments G attacks, so the only freedom left lies in U. Moreover, no argument in U attacks or is attacked by G, since an attacker of a member of G is itself attacked by G by definition of defense, and any argument attacked by G is excluded from U by construction. Attacks into U from arguments that G attacks are neutralised in every complete extension, because G already counterattacks the attacker. Proposition 4.1 states the correctness of the reduction in the form the solver uses. The property follows from known results on complete labellings [2]. A self-contained proof sketch is given in Appendix A.3.

Proposition 4.1 (Grounded-first reduction)**.** Let σ be the complete, the preferred, or the stable semantics. The σ-extensions of AF are exactly the sets $G \cup E$ where E is a σ-extension of the framework restricted to U, with attacks from outside U into U discounted as neutralised.

The exponential enumeration therefore runs only over U instead of over all arguments. In the running example of Figure 2.1, the grounded extension is $\{d, e\}$, the argument c is attacked by it, and the enumeration only has to decide the two-element core $U = \{a, b\}$. The effect on real frameworks is drastic, because the extracted graphs are sparse and their undecided cores are small. On a framework with 24 atoms from an early run, a naive enumeration over all arguments exceeded a 40 second timeout, while the reduction solved the identical framework in 0.32 seconds. Since the enumeration is exponential in $|U|$ and not in the total number of arguments, the solver additionally guards the computation with a bound of $|U| \leq MaxU = 18$, determined empirically from solve times, and falls back to reporting only the grounded extension if the bound is exceeded or a timeout is reached. In the evaluation of Chapter 5, the undecided cores stay at nine or fewer arguments and the guard never triggers. Algorithm 2 in Appendix A.3 summarises the procedure. The solver is parametrised by the framework file and returns its results as JSON, which the Python driver reads back.

4.7 Implementation and Logging

The pipeline is implemented in Python, with the solver in SWI-Prolog [19]. Every run writes a self-contained log directory that makes each reported number traceable to its origin, from the full run configuration and the version identifier of every prompt down to one record per queried argument pair and per LLM call. Appendix A.2 lists the contents in detail.

Two details of this infrastructure matter for the experiments. Prompt versioning ties every result to the exact prompt wording and makes the effect of prompt revisions measurable instead of anecdotal, while the frozen and reusable atom sets are what allow the ablation in Section 5.4 to feed the identical atomized arguments to both extraction variants and the reference annotation of Section 5.5 to be created on exactly the pair set the pipeline judged.

5 Evaluation

This chapter evaluates the pipeline. Section 5.1 describes the experimental setup. Section 5.2 reports the cross-topic results over the eight UKP topics (RQ1, RQ4), and Section 5.3 adds a qualitative check on LLM-generated arguments. Section 5.4 compares typed against binary prompting (RQ2), and Section 5.5 describes the evaluation of the attack quality against a manual reference annotation (RQ3). Section 5.6 closes the chapter with a small case study and an analysis of the recurring error types.

5.1 Experimental Setup

The pipeline is evaluated in two settings. The first starts from argument sets of the UKP Sentential Argument Mining Corpus [18], which provides sentence-level stance annotations for eight controversial topics. Each sentence is labelled `Argument_for`, `Argument_against`, or `NoArgument`. The `NoArgument` sentences are discarded and the test splits are used. Table 5.1 lists the material available per topic. From each topic a balanced sample of 14 arguments, seven pro and seven con, is drawn with the fixed random seed 42, so that the sample is reproducible. The second setting covers the case the pipeline ultimately targets, arguments produced by an LLM, and uses LLM-generated arguments on the topic of allowing autonomous vehicles in public, with ten pro and ten con arguments. Since the generation prompt already constrains each argument to a single claim, atomization is skipped in that setting. The corpus setting carries the quantitative evaluation, because its arguments are not produced by the model under test and therefore provide an independent reference, while the generated setting is a qualitative demonstration that the same pipeline runs on LLM-produced input, as Section 5.3 discusses.

In both settings the extractor and the verifier call GPT-OSS-120B [14], an open-weight mixture-of-experts model with 117 billion parameters, through a university-hosted endpoint at temperature zero, with argument pairs processed in batches of $b = 10$. Candidates are accepted at confidence $c \geq \theta_{\text{acc}} = 70$, forwarded to the verifier at $\theta_{\text{ver}} \leq c < \theta_{\text{acc}}$ with $\theta_{\text{ver}} = 50$, and rejected below θ_{ver}. Since the model is open weight, the experiments do not depend on a proprietary service remaining available or unchanged, and the same weights can in principle be redeployed by anyone.

Tab. 5.1.: Arguments available in the UKP test splits after discarding `NoArgument` sentences. The evaluation samples seven pro and seven con arguments per topic with seed 42.

Topic	Pro	Con	Total
Abortion	136	165	301
Cloning	142	168	310
Death Penalty	103	232	335
Gun Control	158	133	291
Marijuana Legal.	118	126	244
Minimum Wage	116	111	227
Nuclear Energy	122	171	293
School Uniforms	109	146	255

The reproducibility discussion in Section 2.4 shapes the setup in two ways. Atomization is not perfectly deterministic even at temperature zero and produced 21 atoms in one run and 22 in another for the same topic despite identical inputs and seed, so each topic is atomized exactly once and all experiments on it, including both arms of the ablation and the reference annotation, operate on the same frozen atom set. For the same reason, the cross-topic and ablation numbers stem from one canonical batch of runs with one prompt version, and the reference evaluation scores of Section 5.5 from one dedicated run with the final prompt revision, so that no table mixes results from different prompt wordings. The logging infrastructure of Section 4.7 ties every reported number to a concrete run.

5.2 Cross-Topic Results

Table 5.2 reports, for each of the eight UKP topics, the number of atomic arguments after atomization, the number of queried pairs, the number of accepted attacks and their share among all queried pairs, the number of isolated atoms without any incident attack, the size of the grounded extension, the size of the undecided core $|U|$ from Section 4.6, and the numbers of preferred and stable extensions.

Three observations stand out. Every framework contains cycles, a direct consequence of the rebuttal symmetrization described in Section 2.2, since every topic has at least three symmetrized rebuttal edges (column “Sym” of Table 5.3) and each symmetrized pair forms a two-cycle. Five of the eight topics admit several preferred extensions, and the number of stable extensions matches the number of preferred extensions in every case. So although Section 2.1 shows that stable extensions can fail to exist, here every preferred extension is also stable. Each maximal position attacks everything it does not contain, and none of the extracted frameworks behaves like the odd-length

Tab. 5.2.: Cross-topic results over the eight UKP topics. “Atoms” is the number of atomic arguments after atomization, and “Pairs” is the number of queried pairs, excluding pairs of atoms that stem from the same source argument. “Acc” is the number of accepted attacks, that is the edges of the framework, and “Acc%” is their share among all queried pairs. “Iso” counts atoms without incident attacks, “Grnd” is the size of the grounded extension, $|U|$ the size of the undecided core, and #Pr and #St the numbers of preferred and stable extensions.

Topic	Atoms	Pairs	Acc	Acc%	Iso	Grnd	$\lvert U\rvert$	#Pr	#St
Abortion	25	267	41	15.4	6	7	9	4	4
Cloning	20	180	59	32.8	2	9	3	2	2
Death Penalty	18	148	68	45.9	1	3	8	3	3
Gun Control	21	203	26	12.8	5	14	0	1	1
Marijuana Legal.	20	182	41	22.5	2	13	0	1	1
Minimum Wage	22	221	58	26.2	3	10	3	2	2
Nuclear Energy	21	201	43	21.4	1	12	3	2	2
School Uniforms	21	201	29	14.4	3	13	0	1	1

cycle of Figure 2.1, whose undecided conflicts leave no stable extension. These multiple positions are not asserted by a free generation. They are derived from the model’s own pairwise attack judgments through the fixed semantics, and each is conflict-free and admissible by construction. The undecided core, finally, stays small everywhere, with at most nine arguments, which confirms the sparsity assumption behind the grounded-first reduction. On three topics the grounded extension already decides every argument and U is empty, so the formal layer adds nothing beyond the cautious core there. The acceptance rate varies widely, from 12.8 percent for Gun Control to 45.9 percent for Death Penalty, which reflects genuine differences in how directly the sampled arguments engage each other.

A closer look at Abortion, the topic with the most extensions, shows what the multiple positions look like. All four preferred extensions share the grounded core of seven arguments and differ only on the undecided core of nine. Notably, the extensions do not split cleanly along the pro and con labels of the corpus. One extension combines several con arguments about ending abortion through legislation with a pro argument that concedes state regulation after fetal viability, while another accepts only additional pro arguments. The formal layer thus recovers positions that are finer-grained than the binary stance annotation, separating, for instance, a legally framed camp from a morally framed one. At the same time, two of the undecided arguments are defective atoms in the sense of Section 5.6, so part of the multiplicity on this topic rests on atoms that a cleaner atomization would have removed. Both faces of the result, genuinely finer positions and inflation through defective atoms, recur throughout the error analysis.

Table 5.3 breaks the accepted attacks down by type. Undermining is the most common label overall, which fits the corpus, since many UKP sentences report

Tab. 5.3.: Composition of the accepted attacks per topic. “Reb”, “Und”, and “Cut” count edges labelled rebuttal, undermining, and undercutting, including symmetrized reverse edges. “Sym” is the number of edges added by rebuttal symmetrization and is contained in “Reb”. “Ver” counts candidates that were accepted only after verification. “Extract” is the wall-clock extraction time in seconds. The three type counts per topic sum to the Acc column of Table 5.2.

Topic	Reb	Und	Cut	Sym	Ver	Extract (s)
Abortion	30	2	9	15	0	337
Cloning	14	35	10	7	0	161
Death Penalty	22	41	5	11	1	233
Gun Control	6	19	1	3	0	335
Marijuana Legal.	20	6	15	10	1	243
Minimum Wage	6	37	15	3	0	434
Nuclear Energy	12	29	2	6	1	148
School Uniforms	10	17	2	5	3	173

evidence and statistics that target the premises of opposing claims rather than negating their conclusions outright. Undercutting is the rarest label on most topics. The distribution varies strongly by topic. Abortion and Marijuana Legalization are rebuttal-heavy, and on Abortion 15 of the 41 edges exist only through symmetrization, which foreshadows the error analysis in Section 5.6. Verification adds few edges. Across all eight topics, 68 of the 1603 queried pairs were routed to the verifier, which rejected 62 of them and confirmed six, so the extractor’s confidence scores separate clear accepts from clear rejects for about 96 percent of the queried pairs. Extraction takes between two and eight minutes per topic and is dominated by API latency, while solving all three semantics takes under half a second per topic thanks to the grounded-first reduction.

5.3 The LLM-Generated Setting

The second setting feeds twenty LLM-generated arguments on autonomous vehicles through the same extraction and solving stages. Of the 190 queried pairs, 37 are accepted as attacks, an acceptance rate of 19.5 percent that lies within the range of the UKP topics in Table 5.2. The resulting framework is cyclic, yet the grounded extension already accepts or attacks every argument, so all three semantics coincide on a single extension of ten arguments. It contains eight of the ten con arguments but only the two pro arguments that participate in no attack at all, so the extraction judges the con side dominant on this generated argument set. The setting is a qualitative check rather than a second evaluation, since the arguments, the attacks, and the confidence scores all originate from the same model and the run predates the final prompt

revision. It shows that the pipeline runs end-to-end on generated input and that generated argument sets, which are cleaner and more uniform than corpus sentences, tend to be decided completely by the grounded extension.

5.4 Typed versus Binary Prompting

This section measures whether the typed attack prompting of Section 4.3 actually produces different attack relations than plain binary prompting, and what the difference does to the resulting frameworks. A second extraction arm runs the identical frozen atom set $\mathcal{A}'$ of every topic through a binary prompt that asks only whether an attack exists and in which direction, without the attack type, without the type definitions of Section 2.2, and without the two rebuttal rules of Section 4.3. Since the binary arm produces no type labels, no rebuttal symmetrization is possible there. To separate the effect of the categorization from the effect of symmetrization, a third arm is evaluated that takes the typed attack set but drops the symmetrized reverse edges. This arm, called typed-raw, requires no additional extraction, only a second solver run on the reduced edge set.

The overlap of two edge sets E_1 and E_2 is measured by their *Jaccard similarity*

$$J(E_1, E_2) = \frac{|E_1 \cap E_2|}{|E_1 \cup E_2|},$$

which is 1 for identical and 0 for disjoint sets. It is computed twice, once on the directed edges and once on undirected conflict pairs, where each edge is collapsed to the unordered pair of its endpoints. The undirected variant asks the weaker question whether the two arms mark the same pairs as conflicting at all, regardless of the asserted direction.

Table 5.4 summarises the comparison. Three findings emerge.

The two prompts extract substantially different attack sets. The directed Jaccard similarity between the typed-raw and binary edge sets ranges from 0.08 to 0.71 with a mean around 0.4. Even on Death Penalty, the topic with the highest agreement, more than a quarter of the union of both edge sets is found by only one arm. On Abortion the two arms are nearly disjoint, with only five shared edges out of 26 typed-raw and 39 binary edges. The typed arm accepts more raw edges on five of the eight topics, the two arms tie on Gun Control, and the binary arm accepts more on Abortion and School Uniforms. Asking for the attack type therefore changes which conflicts the model asserts rather than rephrasing the same judgment.

Tab. 5.4.: Typed versus binary prompting on identical atom sets. "$|E|$" columns give the number of accepted directed edges for the typed arm, the typed arm without symmetrized edges (raw), and the binary arm. "J" columns give the Jaccard similarity of typed-raw and binary edge sets, directed and undirected. #Pr counts preferred extensions per arm, and the numbers of stable extensions are identical to #Pr in every cell. "Same-st." counts accepted attacks between arguments of the same stance, for the typed arm on the final edge set including symmetrized edges.

	$\lvert E\rvert$			J		#Pr			Same-st.	
Topic	typ.	raw	bin.	dir.	undir.	typ.	raw	bin.	typ.	bin.
Abortion	41	26	39	0.08	0.12	4	1	1	3	14
Cloning	59	52	35	0.47	0.47	2	1	1	7	1
Death Penalty	68	57	51	0.71	0.83	3	1	1	1	2
Gun Control	26	23	23	0.35	0.39	1	1	1	1	1
Marijuana Legal.	41	31	24	0.41	0.53	1	1	1	1	0
Minimum Wage	58	55	49	0.27	0.44	2	1	1	0	10
Nuclear Energy	43	37	27	0.56	0.64	2	1	1	0	0
School Uniforms	29	24	42	0.38	0.38	1	1	1	0	1

Multiple extensions come from symmetrization, not from typing itself. On the final typed frameworks, five of eight topics admit several preferred extensions. On the typed-raw frameworks, every topic collapses to exactly one preferred and one stable extension, the same count as the binary arm. The multiplicity of positions that Section 5.2 reports is thus produced entirely by the mutual attack edges that rebuttal symmetrization adds, a mechanism that is only available when the extraction knows which edges are rebuttals. What this dependence on one modelling decision means for the value of the formal layer is discussed in Section 6.1.

The arms differ in their characteristic false positives. Without a manual reference annotation, edge counts alone cannot say which arm judges more accurately, but the accepted same-stance attacks give a diagnostic signal. Attacks between arguments of the same stance are possible in principle, yet they should be rare in balanced debate samples. The binary arm accepts 14 same-stance attacks on Abortion and 10 on Minimum Wage, among them a cluster in which two statistical claims about abortion frequency attack seven separate contra claims about post-abortion experiences, an implausible pattern within one stance camp. The typed arm accepts far fewer same-stance attacks on those topics, which suggests that forcing the model to name the attack type filters out part of this over-triggering. The signal is not uniform, though. On Cloning the relation reverses, with seven typed against one binary same-stance attack, so the same-stance count is a topic-dependent indicator rather than a general proof of the typed advantage. The typed arm has its own failure mode, as Section 5.6 shows, since several of its Abortion edges attack atoms that are not genuine arguments in the first place. Which of the two

error profiles weighs heavier is exactly the question the manual annotation of the next section is designed to answer.

5.5 Attack Quality against a Manual Reference Annotation

The ablation shows that typed and binary prompting extract different attack sets, but it cannot decide which set is closer to the truth. To answer that, a manually annotated reference set is constructed on the Nuclear Energy topic. The frozen atom set of the canonical run, 21 atoms, yields 201 argument pairs after excluding pairs that share a source argument, exactly the pair set the extraction judges. Every pair is annotated by hand with one of four labels, namely no attack, A attacks B, B attacks A, or mutual attack, following written guidelines based on the attack definitions of Pollock and ASPIC+ [16, 13]. A validation script checks that the annotated pair set matches the queried pair set exactly and that all labels are well-formed. The annotation records only the existence and direction of an attack, not its type. Assigning types by hand would have required premise-level judgments for every pair from a single annotator, and the reference is meant to measure attack detection, on which the typed and the binary arm can be compared directly, rather than type assignment.

The guidelines instruct the annotator to settle the attack question before the direction question and to judge every claim for the object it names as written. A safety claim about one subsystem, for example waste storage, is not read as a claim about the whole nuclear operation, so a conflict that only arises after widening a claim to an object it does not name is labelled as no attack. An attack may rest on a minimal implicit premise, but not on such an object switch. Borderline decisions are recorded in a note column and revisited for consistency before the labels are frozen. The final annotation labels 190 pairs as no attack, 6 pairs as one-directional attacks, and 5 pairs as mutual attacks, which yields 16 directed reference edges and 11 undirected conflict pairs. Genuine conflicts are therefore rare, only about one pair in eighteen carries an attack at all, and this sparsity is worth keeping in mind when reading the scores, since every single reference edge moves recall by six points.

Both extraction arms are then scored against the reference labels, so that the reference annotation simultaneously provides the quality measurement for RQ3 and the missing reliability comparison for RQ2. An extracted attack that the reference annotation confirms is a true positive (TP), an extracted attack that the reference annotation rejects is a false positive (FP), and an annotated attack that the extraction misses is a false negative (FN). The evaluation reports

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}, \qquad \text{F1} = \frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}},$$

that is, the share of extracted attacks that are genuine, the share of genuine attacks that are found, and their harmonic mean. All three are computed on the attack class, both for directed edges and for undirected conflict pairs. Following the metric argument of Section 3.3, accuracy is deliberately not reported. About 46 percent of the 201 pairs connect arguments of the same stance, and since genuine same-stance attacks are rare, a majority of easy negatives would inflate accuracy without saying anything about the quality of the extracted attacks. Precision, recall, and F1 do not contain the true negatives in their formulas, so including the same-stance pairs makes the evaluation stricter rather than more lenient, because every accepted same-stance attack that the reference annotation rejects counts as a false positive.

The scored predictions come from a dedicated evaluation run that executes both extraction arms on the identical frozen atom set with the final revision of the typed and verifier prompts. This revision, made after the ablation batch of Section 5.4, adds an explicit test that two claims which can both be true without tension do not attack each other, whatever their stances, and it requires an undermining attack to name the premise it hits. The typed arm accepts 23 directed edges in this run against 43 in the ablation batch, which is why the edge counts differ from the Nuclear Energy row of Table 5.4. The binary prompt is identical in both batches, and its count still moves from 27 to 32 edges through the run-to-run nondeterminism discussed in Section 5.1. The first difference illustrates the prompt sensitivity and the second the residual nondeterminism that Section 6.2 discusses among the limitations. Scoring happens on three levels. The directed final level compares all accepted edges including the symmetrized reverse edges against the 16 directed reference edges. The directed raw level drops the symmetrized edges and asks how well the model asserts directions on its own. The undirected level collapses edges to conflict pairs and compares them against the 11 reference pairs. Since the binary arm produces no type labels, its raw and final edge sets coincide. Table 5.5 shows the results.

Overall performance. The reference evaluation resolves the question that the ablation left open. The typed arm outperforms the binary arm on every level, with a directed F1 of 0.615 against 0.458. The difference is driven by precision. The binary arm finds every annotated conflict pair, a recall of 1.000 on the undirected level, but it asserts almost three times as many conflict pairs as the reference annotation contains, so the 11 genuine conflicts are buried among 21 spurious ones. The typed arm asserts fewer edges and a larger share of them is genuine. This confirms by direct measurement what the same-stance diagnostic of Section 5.4 could only suggest, namely that requiring the model

Tab. 5.5.: Precision, recall, and F1 of both extraction arms against the manual reference annotation on Nuclear Energy. "n" is the number of predicted edges or pairs, TP the number of true positives. The reference annotation contains 16 directed edges and 11 undirected conflict pairs.

Arm	Level	n	TP	Precision	Recall	F1
typed	directed final	23	12	0.522	0.750	0.615
	directed raw	19	9	0.474	0.562	0.514
	undirected pairs	19	9	0.474	0.818	0.600
binary	directed	32	11	0.344	0.688	0.458
	undirected pairs	32	11	0.344	1.000	0.512

to name an attack type acts as a precision filter. The symmetrized rebuttal edges also earn their place, since they lift the typed directed F1 from 0.514 to 0.615 by supplying reverse directions of mutual conflicts that the model does not assert on its own. The binary arm has no access to this mechanism, and all five of its missed directed edges are reverse directions of mutual reference pairs whose forward direction it does assert.

False positives. The false positives of both arms follow one dominant pattern. The model accepts attacks between claims that belong to different sub-topics of the debate. A representative case is the accepted undermining attack from the atom "building a plant with 100 % security is technically impossible" on the atom "storage is safe and secure". The first claim concerns reactor operation and the second waste storage, so under the strict object reading of the guidelines the pair is compatible, and an attack only arises after widening the security claim to the whole nuclear operation. The same widening connects the water-consumption claims to the claim that nuclear is very competitive once external costs are accounted. These false positives carry confidences up to 90, so raising the acceptance threshold would not remove them without also removing genuine attacks. The verification stage, in contrast, works in the intended direction, since both candidates it rejected in the typed run are pairs that the reference annotation also rejects.

Missed attacks. All four missed directed edges of the typed arm belong to the two mutual reference pairs built on the atom "alternative methods would yield much more climate-saving bang for our buck than nuclear power, according to some analysts". The reference annotation treats this claim as directly contradicting the atoms "nuclear is much cheaper than wind or photovoltaic systems" and "if external costs are accounted, nuclear is very competitive", but the model judges both pairs as compatible in every run. A plausible explanation is the hedged phrasing, which attributes the comparison to unnamed analysts, so the model reads the atom as a report rather than an asserted conclusion. Detecting rebuttals between hedged comparative claims is thus the clearest remaining recall gap.

Stability and context. Three repeated runs of the evaluated configuration on the frozen atom set yield typed directed F1 scores between 0.579 and 0.615, so the typed advantage over the binary arm, sixteen points on the evaluated run, clearly exceeds the run to run variation. For rough context, the scores lie in a similar range as the sentence-level F1 of 67 percent that the knowledge-based system discussed in Section 3.3 reaches on the same corpus, on a task that is related but not directly comparable [9]. In answer to RQ3, the pipeline recovers three quarters of the manually annotated attacks with moderate precision, its errors are systematic rather than random, and the typed extraction arm is measurably closer to the manual annotation than the binary baseline.

5.6 Case Study and Error Analysis

To make the pipeline's behaviour and its typical failures concrete, this section first works through a deliberately small nuclear-energy sample in full detail and then collects the recurring error types that the case study, the ablation, and the reference evaluation surface. The sample is a separate illustrative run with three pro and three con arguments and is not part of the main configuration of Section 5.1, whose frameworks are too large to display as a readable graph. Starting from six balanced UKP arguments, atomization yields eight atomic arguments, three pro and five con. Only the sentence about meltdown statistics is split, into three separate claims, the case already shown in Section 4.2. Of the $\binom{8}{2} = 28$ pairs, 25 are queried after skipping pairs of atoms that share a source argument. Six attacks are accepted. Rebuttal symmetrization adds two reverse edges, so the framework has eight directed attacks, four undermining and four rebuttals.

The graph centers on the argument that nuclear power merits support because it is virtually free of CO_2 emissions, which the con side attacks from several directions. The symmetrized rebuttals make the framework cyclic, so several preferred and stable sets might be expected. In this case, however, the grounded extension already terminates by accepting all five con arguments and rejecting the entire pro side, so grounded, preferred, and stable coincide on a single extension.

Beyond this small sample, three recurring error types stand out across the experiments, each originating in a different stage of the pipeline.

Rebuttal mislabeling creates unjustified cycles. The attack-type classification can mislabel a premise-directed attack as a rebuttal. Since rebuttals are symmetrized into mutual attacks while undermining attacks stay directed, a false rebuttal label introduces a bidirectional conflict and therefore an unjustified cycle. In the case study, the meltdown-fatality claims attack the CO_2 argument as rebuttals, although they question its safety premise rather than directly

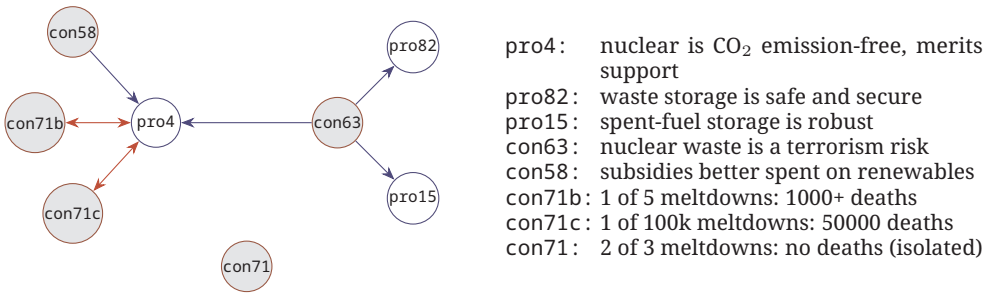

Fig. 5.1.: Attack graph of the small nuclear-energy sample, eight atoms and eight directed edges. Red double arrows are symmetric rebuttals, blue arrows denote undermining attacks. Filled nodes form the grounded extension, which here coincides with the unique preferred and stable extension.

negating its support conclusion. An undermining label would have been more suitable and would not have added a reverse edge. The reference evaluation run of Section 5.5 shows both faces of this mechanism on the same topic. Three of the four symmetrized edges in that run are confirmed by the reference annotation, while the fourth turns a one-directional attack among the cost claims into a spurious mutual conflict and enters Table 5.5 as the false positive with the highest confidence of the run. The cost of this error type scales with the number of symmetrized edges, which Table 5.3 quantifies. Between 3 and 15 edges per topic exist only through symmetrization, and on the rebuttal-heavy Abortion topic more than a third of all edges are symmetrized reverse edges. Every unjustified mutual attack potentially splits the framework into additional extensions, so the multiplicity results of Section 5.2 must be read with this caveat in mind. The descriptive-versus-normative rule described in Section 4.3 was added precisely to curb this error type, and its effect on the numbers is part of the prompt-sensitivity discussion in Section 6.2.

Atomization produces defective atoms. Atomization can produce atoms that, read in isolation, are not self-contained arguments. Three variants occur. Some atoms lose their stance, such as the claim that two out of three meltdowns would cause no deaths, which understates rather than opposes the pro side and stays isolated in the graph. Some atoms keep an unresolved reference despite the self-containment rule, such as the Abortion atom “not all abortions are unjustified according to this argument”. Some atoms are not argumentative at all, such as a sentence that merely cites the introduction of a legislative act. Table 5.2 gives an upper bound on the harmless variant, since between 1 and 6 atoms per topic end up isolated. The harmful variant is when a defective atom does participate in attacks. On Abortion, several edges found only by the typed arm attack exactly the two defective atoms above, so extraction errors and atomization errors compound.

Binary prompting over-triggers within one stance camp. The third error type is specific to the binary arm. Without the type definitions, the model accepts attacks between same-stance arguments far more liberally, most visibly in the Abortion cluster already described in Section 5.4, a pattern that the typed prompt largely suppresses on the affected topics. This asymmetry of error profiles, spurious cycles and defective targets on the typed side against same-stance over-triggering on the binary side, is why edge counts alone could not settle the comparison, and why the verdict of Section 5.5 rests on the manual annotation rather than on a metric chosen in either arm's favour.

6 Discussion

6.1 The Added Value of the Formal Layer

How much the formal layer adds varies strongly across the eight topics of Section 5.2. The clearest payoff appears on Abortion, where four preferred extensions over an undecided core of nine arguments make competing internally consistent positions explicit, and on Death Penalty with three extensions over a core of eight. On Gun Control, Marijuana Legalization, and School Uniforms, by contrast, the undecided core is empty, the grounded extension already decides every argument, and the enumeration adds nothing beyond the cautious core. That every extracted framework is cyclic is a precondition for this payoff rather than a merit in itself, since in an acyclic framework the grounded, preferred, and stable semantics coincide on a single extension [6]. An honest reading is therefore that the formal layer is most useful where the debate genuinely splits into competing coherent camps, and closer to redundant where one side dominates the sampled arguments.

The ablation sharpens this picture. The formal richness that the pipeline reports is not a free observation about the debates but a consequence of one deliberate modelling decision, the symmetrization of rebuttals, which only typed extraction makes available. This is a feature and a caveat at once. It is a feature because the mutual attack is the semantically correct rendering of a genuine contradiction between conclusions. It is a caveat because every mislabelled rebuttal buys an unjustified cycle, and Section 5.6 shows that such mislabelling occurs. The computational price of the richness stays low. The undecided cores remain at nine or fewer arguments, and all three semantics solve in under half a second per topic.

Read against the motivation of Chapter 1, the multiple extensions are more than a structural statistic. A standalone LLM that is asked to summarise the positions of a debate returns another fluent generation without a guarantee of internal consistency. The extensions computed here carry that guarantee by construction, since every preferred extension is conflict-free and admissible, and every acceptance is traceable to the extracted attack edges that produce it. On Abortion the difference becomes concrete, since the four extensions separate a legally framed camp from a morally framed one, a finer distinction than the binary stance labels of the corpus. The formal layer thus delivers what a raw model answer leaves implicit, a set of debate positions whose joint acceptability is checkable rather than asserted.

These results also feed back into the related work of Chapter 3. They supply the empirical layer that ARGUS [3] leaves open, showing that such a pipeline produces sparse and cyclic frameworks with small undecided cores rather than intractable graphs. They refine the finding of Gorur et al. [8] that LLMs identify attack relations reliably, since the reference evaluation locates the benefit of typed prompting in precision rather than recall. And they qualify the quantitative route of ArgLLMs [7], because a single strength value per argument could not have represented the four incompatible Abortion positions that the set-based semantics make explicit.

6.2 Limitations

Four limitations qualify the results, namely the sensitivity of the extraction to prompt wording, the small samples, the possible familiarity of the model with the corpus, and the reliance on a single model and a single annotator.

Prompt sensitivity. The extraction is sensitive to the exact wording of the prompts. When the attack prompt was revised during development, among other things to add the descriptive-versus-normative rule, the accepted edge sets of otherwise identical runs changed substantially, on some topics by dozens of edges, and the number of extensions changed with them. The final revision before the reference evaluation run, which tightened all three type definitions, roughly halved the accepted typed edges on Nuclear Energy from 43 to 23. The cross-topic and ablation numbers therefore come from one canonical batch with one versioned prompt, the reference scores from one dedicated run with the final revision, and all of them should be read as results about the pipeline with these exact prompts, not about typed prompting in the abstract. The prompt versioning of Section 4.7 makes this dependency explicit and traceable rather than removing it.

Sampling and scale. The UKP samples are small, with seven pro and seven con arguments per topic, which limits how far the cross-topic numbers generalise to larger argument sets. The residual nondeterminism of the atomization, described in Section 5.1, is controlled by the frozen atom sets within this evaluation, but it means that an independent reproduction will operate on a slightly different atom set.

Data contamination. The UKP corpus has been publicly available since 2018 and may therefore have been part of the training data of GPT-OSS-120B, whose exact composition is not disclosed. In that case the model may recognise the arguments and their stances rather than judge them from scratch. The pairwise attack judgments, however, cannot be memorised. The reference

annotation of Section 5.5 was created for this thesis on a frozen atom set and is unpublished, and the atomized claims the model judges mostly differ in wording from the corpus sentences. Familiarity with the arguments could still make the task easier than it would be on genuinely unseen debates, so the absolute scores should be read with this reservation.

Model and self-evaluation. All results stem from a single model at temperature zero. In the LLM-generated setting the arguments, the attacks, and the confidence scores all come from the same model, so there is no independent quality signal, and that setting is treated as a qualitative check rather than a second evaluation. The reference annotation of Section 5.5 is created by a single annotator, the author, and no inter-annotator agreement can be reported, a limitation that is common for a thesis but should be kept in mind when reading the scores. The same person also designed the pipeline and its prompts, so annotation decisions and extraction behaviour could share blind spots, which the written guidelines and the strict object reading mitigate but cannot rule out. The reference set is moreover small, with 16 directed edges on a single topic, so every individual edge moves recall by six points and small F1 differences between configurations should not be over-interpreted.

6.3 Design Trade-offs

Four design decisions shape the frameworks and deserve explicit discussion.

The symmetrization of rebuttals is well motivated by the absence of a preference ordering in Dung's abstract framework, but as Section 6.1 discusses, it is also the single source of every multi-extension result, which makes a mislabelled rebuttal costly. On the manually annotated topic the mechanism proves its worth, since three of the four reverse edges it adds are confirmed by the reference annotation. A natural refinement would be to symmetrize only above a confidence threshold, or to let the verifier confirm specifically the mutuality of the contradiction rather than the attack as such.

Dropping the attack type when converting to binary attack facts is consistent with the set-based question Dung semantics answer, yet it means the pipeline extracts information it then discards. The types, confidence scores, and justifications are retained as metadata, and this thesis itself demonstrates their analytical value, since the error analysis and the ablation both rely on them. A formalism that consumes them in the semantics, such as ASPIC+ or a weighted framework, is the obvious next step and is discussed as future work.

The confidence thresholds are design choices rather than learned values, since the score is a routing signal and not a calibrated probability. The logged decisions of Section 5.2 show that the verification stage acts almost exclusively

as an additional reject filter, since it confirmed only six of the 68 borderline candidates across the canonical batch, and the reference evaluation run of Section 5.5 gives a first, small-sample indication that this filtering points in the right direction. Whether the six rescued accepts justify the extra LLM call per borderline case is a question these few cases cannot settle. Now that a reference annotation exists, a natural follow-up would be to tune both thresholds against it, for instance to maximise the F1 score, which was ruled out at design time because no labels were available and tuning on the evaluation data would have leaked information.

The exhaustive pairwise querying, finally, bounds the scale the pipeline can handle. The number of queried pairs grows quadratically with the atom count. The 21 Nuclear Energy atoms already produce 201 pairs, and extraction takes between two and eight minutes per topic at this size, so a realistic debate with hundreds of arguments would produce tens of thousands of pairs and hours of extraction. Applying the pipeline at that scale requires a cheaper candidate filter before the LLM call, for example an embedding-based prescreening that discards clearly unrelated pairs, at the price of missing conflicts the filter does not anticipate.

7 Conclusion

This thesis presented an end-to-end pipeline that turns natural-language debate text into a formally evaluated Dung argumentation framework, from typed attack extraction through confidence filtering to a Prolog solver under grounded, preferred, and stable semantics.

Returning to the research questions, RQ1 is answered in the affirmative. The pipeline runs end-to-end across eight UKP topics and an LLM-generated setting, and it produces well-defined frameworks whose extensions are computed in a fraction of a second per topic thanks to the grounded-first reduction, since the undecided cores never exceeded nine arguments in the evaluation.

RQ2 is answered by the ablation. Typed and binary prompting extract substantially different attack sets on identical atomized arguments, with directed Jaccard similarities between 0.08 and 0.71, so the categorization changes what the model asserts rather than rephrasing it. The structural effect on the frameworks, however, runs entirely through rebuttal symmetrization. Without the symmetrized edges, typed and binary frameworks alike collapse to a single extension on every topic. Typing also changes the error profile, since on the affected topics it suppresses most of the implausible same-stance attacks that the binary prompt accepts, while introducing failure modes of its own. The evaluation against the manual reference annotation confirms that this filtering pays off in accuracy.

RQ3 is answered by the reference evaluation on the frozen Nuclear Energy atom set. The typed arm recovers twelve of the sixteen manually annotated directed attacks at a precision of 0.522, a directed F1 of 0.615 against 0.458 for the binary arm, with a spread of 0.579 to 0.615 across three repeated runs of the same configuration. The errors are systematic rather than random. The false positives assert attacks across sub-topic boundaries that the strict object reading of the annotation guidelines excludes, and the missed attacks concentrate on hedged contradictions. Typed extraction is thus measurably closer to the manual annotation than the binary baseline.

RQ4 is answered by the cross-topic analysis. Five of eight topics admit several preferred and stable extensions, and the formal layer makes explicit several internally consistent positions that the raw argument set leaves implicit. The advantage of the enumeration over the grounded extension varies across topics, since on three topics the grounded extension already decides every argument.

Overall the results demonstrate the feasibility of integrating modern language models with established formal argumentation, and they suggest that formal

semantics can provide a transparent reasoning layer on top of LLM-generated argumentative content, provided that the modelling decisions which create the formal structure, above all the treatment of rebuttals, are made explicit and kept under experimental control.

7.1 Future Work

Several directions remain open. The most immediate one is to broaden the empirical basis. The extraction quality should be evaluated against annotated datasets beyond the single-topic manual annotation of this thesis, ideally with a second annotator so that inter-annotator agreement can be reported, and the experiments should be repeated across several models to separate pipeline-specific from model-specific behaviour.

A second direction deepens the symbolic reasoning side. The typed attacks and confidence scores, which the current pipeline retains only as metadata outside the formal semantics, could be consumed directly by a richer formalism such as ASPIC+ or a weighted bipolar framework, and the symmetrization decision could be made confidence-dependent instead of unconditional.

A third direction concerns the pipeline in use. The exhaustive pairwise querying bounds it to small argument sets, so scaling to realistic debates with hundreds of arguments depends on the cheaper candidate prescreening sketched in Section 6.3, whose price in missed conflicts would itself have to be measured. The pipeline should also be compared qualitatively against a standalone LLM that is asked to summarise the coherent positions of a debate, which would test whether the formal extensions capture positions that free generation misses.

Bibliography

[1]Gianvito Alfano et al. *LLM-based Argument Mining meets Argumentation and Description Logics*. 2026. arXiv: 2603.02858 (cit. on p. 13).

[2]Pietro Baroni, Martin Caminada, and Massimiliano Giacomin. “An introduction to argumentation semantics”. In: *The Knowledge Engineering Review* 26.4 (2011), pp. 365–410 (cit. on pp. 19, 20).

[3]Ettore Caputo, Sergio Greco, and Lucio La Cava. “ARGUS: Towards End-to-End Argument Mining with Large Language Models”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 40. 48. 2026, pp. 41547–41549 (cit. on pp. 11, 36).

[4]Federico Cerutti, Sarah Alice Gaggl, Matthias Thimm, and Johannes Peter Wallner. “Foundations of Implementations for Formal Argumentation”. In: *Journal of Applied Logics* 4.8 (2017), pp. 2623–2705 (cit. on p. 19).

[5]Günther Charwat, Wolfgang Dvořák, Sarah Alice Gaggl, Johannes Peter Wallner, and Stefan Woltran. “Methods for solving reasoning problems in abstract argumentation – A survey”. In: *Artificial Intelligence* 220 (2015), pp. 28–63 (cit. on p. 19).

[6]Phan Minh Dung. “On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games”. In: *Artificial Intelligence* 77.2 (1995), pp. 321–357 (cit. on pp. 2, 5, 6, 35).

[7]Gabriel Freedman, Adam Dejl, Deniz Gorur, et al. “Argumentative large language models for explainable and contestable claim verification”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 14. 2025, pp. 14930–14939 (cit. on pp. 11, 36).

[8]Deniz Gorur, Antonio Rago, and Francesca Toni. “Can Large Language Models perform Relation-based Argument Mining?” In: *Proceedings of the 31st International Conference on Computational Linguistics*. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 8518–8534 (cit. on pp. 12, 36).

[9]Stefan Kramer et al. *Focusing knowledge-based graph argument mining via topic modeling*. 2021. arXiv: 2102.02086 (cit. on pp. 12, 32).

[10]Paul Krause, Simon Ambler, Morten Elvang-Goransson, and John Fox. “A logic of argumentation for reasoning under uncertainty”. In: *Computational Intelligence* 11.1 (1995), pp. 113–131 (cit. on p. 1).

[11]John Lawrence and Chris Reed. “Argument mining: A survey”. In: *Computational linguistics* 45.4 (2019), pp. 765–818 (cit. on p. 8).

[12]Hao Li, Viktor Schlegel, Yizheng Sun, Riza Batista-Navarro, and Goran Nenadic. "Large language models in argument mining: A survey". In: *arXiv preprint arXiv:2506.16383* (2025) (cit. on p. 8).

[13]Sanjay Modgil and Henry Prakken. "The ASPIC$^+$ framework for structured argumentation: A tutorial". In: *Argument & Computation* 5.1 (2014), pp. 31–62 (cit. on pp. 7, 8, 13, 29).

[14]OpenAI. *gpt-oss-120b & gpt-oss-20b Model Card*. 2025. arXiv: 2508.10925 [cs.CL] (cit. on pp. 23, 51).

[15]Paulo Pirozelli, Victor Hugo Nascimento Rocha, Fabio G. Cozman, and Douglas Aldred. *An LLM-Based System for Argument Mining*. 2026. arXiv: 2605.13793 [cs.CL] (cit. on p. 11).

[16]John L. Pollock. "How to reason defeasibly". In: *Artificial Intelligence* 57.1 (1992), pp. 1–42 (cit. on pp. 7, 29).

[17]Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting". In: *The Twelfth International Conference on Learning Representations (ICLR)*. 2024 (cit. on p. 8).

[18]Christian Stab, Tristan Miller, Benjamin Schiller, Pranav Rai, and Iryna Gurevych. "Cross-topic Argument Mining from Heterogeneous Sources". In: *Proceedings of EMNLP 2018*. 2018, pp. 3664–3674 (cit. on pp. 3, 12, 23).

[19]Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. "SWI-Prolog". In: *Theory and Practice of Logic Programming* 12.1-2 (2012), pp. 67–96 (cit. on p. 20).

List of Figures

List of Tables

A Prompts, Reproducibility, and Proof Details

A.1 Prompts

This appendix lists the prompts of the final configuration verbatim, in the versions used by the reference evaluation run of Section 5.5 (`classify_batch_v4` and `verify_v4`). Placeholders in curly braces are filled by the pipeline at run time. The typed classification prompt and the verifier prompt contain the two rebuttal rules discussed in Section 4.3 as well as the general not-an-attack rule of the final revision. The cross-topic and ablation batch of Sections 5.2 and 5.4 used the preceding version (`classify_batch_v3` and `verify_v3`), which states the stance rule only inside the rebuttal definition and lacks the general not-an-attack test and the specificity requirements for undercutting and undermining. The binary prompt (`classify_batch_binary_v1`) is identical in both batches and omits the attack type, the type definitions, and these rules.

Atomization prompt

```
You receive an argument from a debate on "{topic}".
Check whether it bundles multiple distinct claims into one sentence.
If yes, split it into atomic arguments (one claim each), preserving the stance.
If it already expresses a single claim, return it unchanged.

Important: Only promote claims the speaker actually ASSERTS as their own.
- Concessive/contrastive clauses (markers: "but", "although", "while", "despite", "even though", "yet") are rhetorical hedges, NOT independent
  arguments. Keep them attached to the main claim or drop them - never make
  them a separate atom.
- Reported or conceded positions of the opposing side (e.g. "opponents argue
  X", "greens always argue X", "some claim X, but ...") are NOT the speaker's
  own claim. Do NOT promote them to a separate atom; drop them.
This prevents atoms that, read in isolation, carry the OPPOSITE stance of the
speaker (e.g. a pro-nuclear sentence must never yield a pro-renewable atom).

```

```
Self-containment rule: Every output argument MUST be understandable without
any surrounding context. If a fragment contains pronouns like "it", "them",
"this", "they" without a clear referent, replace the pronoun with the explicit
noun from the original sentence. If the referent cannot be recovered, drop
the fragment entirely.

Argument [{id}] ({stance}):
{text}

Respond ONLY with a JSON array, no markdown. Use IDs like "{id}", "{id}b", ...:
[
  {"id": "{id}",  "text": "...", "stance": "{stance}"},
  {"id": "{id}b", "text": "...", "stance": "{stance}"}
]
```

Typed classification prompt (batched)

```
Propose attack candidates for the following argument pairs from a debate on
"{topic}".

An attack exists when one argument:
- asserts a conclusion logically incompatible with the other's conclusion -
  the one claim can be true only if the other is false (rebuttal), OR
- undermines a premise the other relies on (undermining), OR
- shows the other's conclusion does not follow even if its premise is true
  (undercutting)

NOT an attack - this applies to ALL three types: merely taking the opposing
stance, addressing a different sub-topic, or generally "arguing for the other
side". A con argument does not attack every pro argument (and vice versa).
Test: if both specific claims can be true at the same time without tension,
there is NO attack and direction MUST be NONE.

{pairs}

direction    = exactly one of: A_ATTACKS_B / B_ATTACKS_A / NONE
attack_type  = one of: rebuttal / undercutting / undermining / NONE
  rebuttal:     the two conclusions are logically incompatible - they negate
                the SAME proposition, so one claim is true only if the other
                is false; there must be a direct contradiction between the two
```

```
                specific claims. A descriptive/empirical claim (what people
                believe or do, statistics, trends) and a normative claim (what
                ought to be) are NOT logically incompatible - they cannot
                rebut each other (at most undermining/undercutting). A
                rebuttal is inherently MUTUAL (it holds in both directions);
                pick either direction.
  undercutting: attacker shows the target's conclusion doesn't follow even if
                the target's premise holds - it targets the INFERENCE of that
                specific argument. A general counter-consideration on the same
                topic is NOT undercutting.
  undermining:  attacker directly attacks a premise the target relies on. The
                premise must be a SPECIFIC claim the target states or
                presupposes - the target's overall stance or the general
                attractiveness of its position is NOT a premise. In "reason",
                name the attacked premise; if you cannot name it, the type is
                not undermining.
  If direction=NONE, attack_type MUST be NONE.
confidence   = integer 0-100: probability that ANY attack exists between the
  two arguments (0 = definitely no attack, 100 = definitely attack)
  If direction=NONE, confidence MUST be 0.

Respond ONLY with a JSON array, same order as the pairs above, no markdown:
[
  {"pair": "id_a:id_b", "direction": "A_ATTACKS_B", "attack_type": "rebuttal",
   "confidence": 85, "reason": "one sentence"},
  ...
]
```

Binary classification prompt (batched, ablation arm)

```
Propose attack candidates for the following argument pairs from a debate on
"{topic}".

An attack exists when one argument is logically incompatible with the other's
conclusion, undermines a premise the other relies on, or shows that the
other's conclusion does not follow even if its premise holds. Merely taking
the opposing stance, addressing a different sub-topic, or arguing for the
```

```
other side is NOT enough. There must be a genuine conflict between the two
specific claims.

{pairs}

direction    = exactly one of: A_ATTACKS_B / B_ATTACKS_A / NONE
confidence   = integer 0-100: probability that ANY attack exists between the
  two arguments (0 = definitely no attack, 100 = definitely attack)
  If direction=NONE, confidence MUST be 0.

Respond ONLY with a JSON array, same order as the pairs above, no markdown:
[
  {"pair": "id_a:id_b", "direction": "A_ATTACKS_B", "confidence": 85,
   "reason": "one sentence"},
  ...
]
```

Verifier prompt (typed)

```
Two arguments from a debate on "{topic}":

[{id_a}] ({stance_a}) {text_a}
[{id_b}] ({stance_b}) {text_b}

A previous analysis claims: {claimed}

Does this attack relation genuinely hold?
Does A undermine a premise B relies on (undermining)? The premise must be a
SPECIFIC claim B states or presupposes - B's overall stance or the general
attractiveness of its position is NOT a premise. If you cannot name the
attacked premise, it is not undermining.
Do A and B assert logically incompatible conclusions that negate the same
proposition, so that one is true only if the other is false (rebuttal)?
Opposing stance alone is NOT enough.
A descriptive claim (what people believe/do, statistics) and a normative claim
(what ought to be) are NOT logically incompatible and cannot rebut each other.
Does A show B's conclusion doesn't follow even if B's premise is true
(undercutting)?
A general counter-consideration on the same topic is NOT undercutting.
Decisive test: if both specific claims can be true at the same time without
tension, there is NO attack - merely being on opposite sides of the debate is
not enough.
If any of the three genuinely holds, confirm=true.

Respond ONLY with JSON, no markdown:
```

```
{"confirms": true, "reason": "one sentence"}
```

A.2 Reproducibility

All experiments use the following canonical configuration. The model is GPT-OSS-120B [14], called through the university-hosted inference endpoint at temperature zero with a batch size of ten pairs per classification call. Per topic, 14 arguments are sampled from the UKP test split, seven pro and seven con, with random seed 42. Candidates are accepted at confidence 70 or above, verified between 50 and 69, and rejected below 50. Rebuttal symmetrization is enabled in the typed arm and structurally impossible in the binary arm. The solver computes preferred and stable extensions with the grounded-first reduction and skips them if the undecided core exceeds 18 arguments, which did not occur in the evaluation.

Every run writes a log directory containing the run configuration, the frozen atomized argument set, one record per queried pair with the model output and routing outcome, the accepted attacks, the generated Prolog facts, and the solver results. The ablation additionally stores, per topic, the shared atom set and one full run log per extraction arm, together with a comparison file holding the edge sets and Jaccard values. All tables in Chapter 5 are generated by a single script from these artifacts, so that every reported number is reproducible from the logged runs without further API calls.

The complete implementation, the run logs of all reported experiments, the reference annotation, and the table generation script are available from the author on request. The UKP corpus itself is not redistributed and must be obtained from its original authors under their license agreement.

A.3 Algorithm and Proof Sketch for the Grounded-First Reduction

Algorithm 2 summarises the grounded-first solving procedure of Section 4.6.

The rest of this section gives the proof sketch for Proposition 4.1, which states that the σ-extensions of AF for $\sigma \in \{\text{complete}, \text{preferred}, \text{stable}\}$ are exactly the sets $G \cup E$ where E is a σ-extension of the framework restricted to the undecided core U.

Proof sketch. Let S be a complete extension of AF. Since the grounded extension is the least complete extension, $G \subseteq S$, and conflict-freeness excludes from S every argument attacked by G, so $S = G \cup E$ for some $E \subseteq U$. By definition of U, no attacker of an argument in U lies in G, so every attacker of

Algorithm 2 Grounded-first solving with undecided-core guard

Require: framework $AF = \langle \mathcal{A}, \mathcal{R} \rangle$, bound $MaxU$
Ensure: grounded extension G, preferred set $\mathcal{P}$, stable set $\mathcal{S}$

$G \leftarrow \emptyset$
repeat ▷ fixed-point iteration, polynomial
  $G \leftarrow F_{AF}(G)$
until G unchanged
$U \leftarrow \{x \in \mathcal{A} \mid x \notin G$ and no $g \in G$ attacks $x\}$
if $|U| > MaxU$ **then**
  return $(G, \bot, \bot)$ ▷ enumeration skipped, only grounded reported
end if
$\mathcal{P}_U \leftarrow$ maximal admissible subsets of $AF|_U$, with attacks from outside U discounted as neutralised
$\mathcal{S}_U \leftarrow$ conflict-free $E \subseteq U$ that attack every $x \in U \setminus E$
return $(G,\ \{G \cup E \mid E \in \mathcal{P}_U\},\ \{G \cup E \mid E \in \mathcal{S}_U\})$

an argument in E is either attacked by G, and hence neutralised in S, or lies in U itself. Defense within S therefore coincides with defense within the discounted restriction to U. In the other direction, $G \cup E$ is conflict-free for every conflict-free $E \subseteq U$, because an attacker of a member of G is itself attacked by G and therefore lies outside U, and the same case analysis shows that $G \cup E$ is complete in AF whenever E is complete in the restriction. Together, $S \mapsto S \setminus G$ is a one-to-one correspondence between the complete extensions of AF and those of the restriction. Preferred extensions are the maximal complete ones, and the correspondence preserves inclusion, which settles the preferred case. For the stable case, an argument outside $G \cup E$ is either attacked by G or lies in $U \setminus E$, where stability of E in the restriction requires an attack from E, so $G \cup E$ attacks every outside argument, and the converse direction follows by restricting a stable extension of AF to U. □

Colophon

This thesis was typeset with LaTeX 2$_\varepsilon$. It uses the *Clean Thesis* style developed by Ricardo Langner. The design of the *Clean Thesis* style is inspired by user guide documents from Apple Inc.

Download the *Clean Thesis* style at `http://cleanthesis.der-ric.de/`.

Adaptations to the style of the Institute of Computer Science can be found at `https://gitlab.rlp.net/institut-fur-informatik/cleanthesis-jgu`.

Eigenständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe (dazu zählen auch „KI-Werkzeuge“). Sämtliche wörtlichen und sinngemäßen Übernahmen und Zitate sind kenntlich gemacht und nachgewiesen. Auch beim Einsatz von KI-Werkzeugen[1] stellt die Arbeit klar dar, in welchem Umfang diese genutzt wurden und welche Inhalte dadurch erzeugt oder beeinflusst wurden[2]. Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferinnen und Prüfer explizit ausgeschlossen haben.

Mit Abgabe der vorliegenden Leistung übernehme ich die Verantwortung für das eingereichte Gesamtprodukt. Ich verantworte damit auch alle KI-generierten Inhalte, die ich in meine Arbeit übernommen habe. Die Richtigkeit übernommener (KI-generierter) Aussagen und Inhalte habe ich nach bestem Wissen und Gewissen geprüft.

Ich habe die Arbeit nicht zum Erwerb eines anderen Leistungsnachweises in gleicher oder ähnlicher Form eingereicht. Von der Ordnung zur Sicherung guter wissenschaftlicher Praxis und zum Umgang mit wissenschaftlichem Fehlverhalten habe ich Kenntnis genommen.

Mir ist bekannt, dass ein Verstoß gegen die genannten Punkte prüfungsrechtliche Konsequenzen hat und insbesondere dazu führen kann, dass die Studien- und Prüfungsleistung als mit „nicht bestanden“ bewertet wird. Die Einschreibung kann für bis zu zwei Jahre widerrufen werden, wenn Studierende zweimal oder häufiger bei Prüfungsleistungen täuschen (§ 69 Abs. 4 und 5 HochSchG).

Mainz, July 15, 2026

Aaron Habyarimana

[1] Mit „KI-Werkzeugen“ werden computergestützte Systeme bezeichnet, die auf Basis maschinellen Lernens neue Inhalte generieren können (z. B. ChatGPT, Gemini, Claude, LLaMA, Midjourney, Stable Diffusion o. ä.).

[2] Die Nutzung von KI-Werkzeugen muss dann kenntlich gemacht werden, wenn Sie den Kern der Aufgabenstellung in von den Prüfern nicht anzunehmender Weise berührt: Sollte nichts anderes explizit mit den Prüferinnen und Prüfern vereinbart worden sein, so ist dies der Fall, sobald wesentliche Teile der eingereichten Arbeit (z. B. ganze Sätze des Textes, ganze Abbildungen, nicht-offensichtliche Teile von Rechnungen oder Beweisen, mehrere Zeilen von Programmcode am Stück) von solchen Werkzeugen generiert wurden und wörtlich oder in abgewandelter Form in die Arbeit übernommen wurden oder wesentliche Konzepte oder Ideen (z. B. eine Gliederung der Literatur oder ein Lösungsansatz für ein Problem) von KI-Werkzeugen generiert wurden.

Hinweis: Es wird im Falle von abweichenden Vereinbarungen mit den Prüferinnen und Prüfern empfohlen, jene im Vorfeld schriftlich festzuhalten und zusätzlich auch noch einmal in der eingereichten Arbeit zu benennen.

Use of AI tools.

AI Tool	Used for	Reason	When
Claude (Anthropic)	Implementation and debugging of the pipeline and evaluation scripts (Python, Prolog), including TikZ code for thesis figures	Support with implementation and figure generation	Implementation and experimentation phase
Claude (Anthropic)	Drafting, language editing, and restructuring of the thesis text	Writing assistance	Writing phase, entire thesis